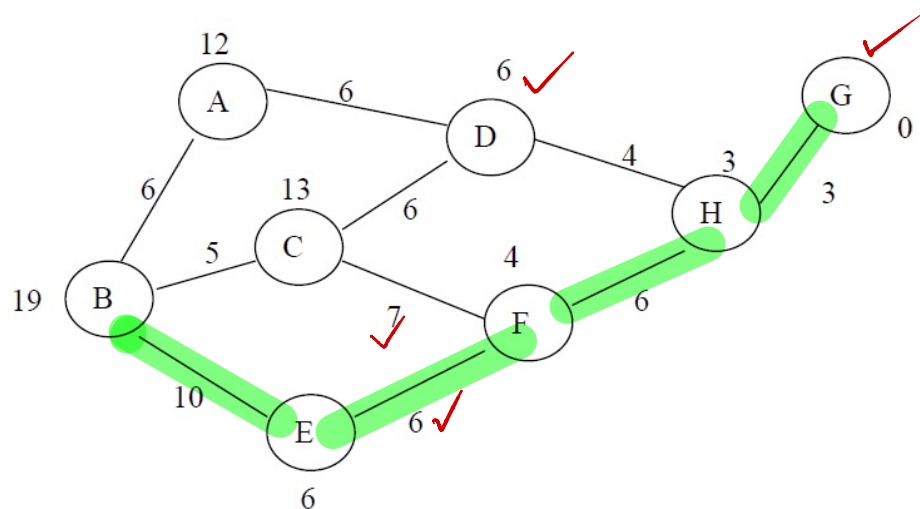


1.

- (a) Calculate the path that takes the shortest time to travel from the **B** node to the **G** node, using the A* algorithm.
Show your working in the table provided.



Start node	Destination node	Cost from start node (g)	Heuristic (h)	Total (f = g + h)
B	B	0	19	19
B	A	6	12	18
B	C	5	13	18
B	E	10	6	16
E	F	10 + 6 = 16	4	20
F	C	16 + 7 = 23	13	36
F	H	16 + 6 = 22	3	25
H	D	22 + 4 = 26	6	32
H	G	22 + 3 = 25	0	25

Final Path B → E → F → H → G.

[5]

(b) Describe the purpose of both the A* algorithm and Dijkstra's algorithm. [2]

A* algorithm: it is designed to find the shortest and most efficient path between a starting node and a specific goal node. It uses both the actual cost from the start and an estimated cost here, stick to the goal to make smarter decisions while searching.

Dijkstra's algorithm: it finds the shortest path from a single starting node to all other nodes in a weighted graph. It explores all possible parts systematically without making any assumptions or guesses.

2. The table shows part of the instruction set for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC.
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store the contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to the ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
CMP	#n	Compare the contents of ACC with number n.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
AND	#n	Bitwise AND operation of the contents of ACC with the operand.
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>.
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand.
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>.
OR	#n	Bitwise OR operation of the contents of ACC with the operand.
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>. <address> can be an absolute address or a symbolic address.
LSL	#n	Bits in ACC are shifted n places to the left. Zeros are introduced on the right hand end.
LSR	#n	Bits in ACC are shifted n places to the right. Zeros are introduced on the left hand end.
IN		Key in a character and store its ASCII value in ACC.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

CS 9618 - Mock 2025

Write the assembly language program code that represents the following pseudocode algorithm.

```

COUNT ← 0 ✓
OUTPUT B
WHILE COUNT <> 5
    OUTPUT B
    COUNT ← COUNT + 1
ENDWHILE
OUTPUT A
END
    
```

$\frac{O/P}{0}$ $\frac{ACC}{0}$
 0 0

Label	Op code	Operand	Comment
	LDM	#0	// load 0
	STO	COUNT	// store in COUNT
	LDD	B	// load B
	OUT		// output B
LOOP:	LDD	COUNT	// load COUNT
	CMP	#5	// is COUNT = 5 ?
	JPE	ENDLOOP	// jump to ENDLOOP if TRUE
	LDD	B	// load B
	OUT		// output B
	LDD	COUNT	// load COUNT
	INC	ACC	// increment ACC
	STO	COUNT	// store in COUNT
	JMP	LOOP	// jump to LOOP
ENDLOOP:	LDD	A	// load A
	OUT		// output A
	BND		// end program
A:	1		
B:	0 ✓		
COUNT:	1		

[4]

3.

- (a) Given the following code snippet, analyze the order of time complexity and explain your reasoning. Assign the appropriate Big O notation to represent the time complexity.

Code Snippet:

```
FOR i ← 1 TO n
  FOR j ← 1 TO n
    OUTPUT "Hello, ZAK!"
  NEXT
NEXT
```

Big O notation: $O(n^2)$

Brief explanation:

The code contains two nested loops.

The outer loop runs n times, and for each iteration of the outer loop the inner loop also runs n times.

The total number of iterations is proportional to N multiply N is equal to N square.

As a result, the complexity is $O(n^2)$ which is quadratic time. This means that if the input size doubles, the number of operations roughly increases four times. [3]

- (b) Compare the performance of the algorithms for a binary search and a linear search using Big O notation for order of time complexity.

Linear search: it checks each element one by one until the required element is found or the list ends. The time complexity is $O(n)$ which is constant because in the worst case, all an elements must be checked.

Binary search: divides the search space into halves each time by comparing the middle element with the target. The complexity is $O(\log n)$ because the list is half with each comparison. [3]

4. An international conference categorizes its submissions using codes generated according to specific rules. These rules are defined using the Backus-Naur Form (BNF), outlined as follows:

 $\checkmark$ $\langle \text{Category} \rangle ::= A|B|C|D|E$

 $\langle \text{Year} \rangle ::= '20' \langle \text{Digit} \rangle \langle \text{Digit} \rangle$

 $\langle \text{Digit} \rangle ::= 0|1|2|3|4|5|6|7|8|9$

 $\langle \text{SubmissionCode} \rangle ::= \langle \text{Category} \rangle \langle \text{Year} \rangle \langle \text{Digit} \rangle \langle \text{Digit} \rangle \langle \text{Digit} \rangle$

- (a) Create a definition for SubmissionCode as a record structure UDT, including components for Category, Year, and a sequence number.:

Record SubmissionCode

DECLARE Category: (A,B,C,D,E)

DECLARE Year: STRING

DECLARE SequenceNumber: INTEGER

End Record

[2]

(b) State whether each of the following is a valid code or an invalid code. In the case of an invalid code, explain how the BNF rules given in question 4. are broken.

(i) A2023012 [2]

.....

(ii) C201875 [2]

.....

(c) The conference introduces a new code system to differentiate submissions.
 Alter and/or extend the BNF definition given in 4 above to:

Define <ExtendedSubmissionCode> that includes an **optional** abstract prefix
 "ABS-" before the <Category>. [4]

.....
 $\langle x \rangle ::= \text{'ABS-'} \langle \text{Category} \rangle \mid \langle \text{Category} \rangle$

.....
 $\langle \text{extendedSubmissionCode} \rangle ::= \langle x \rangle \langle \text{year} \rangle \langle \text{digit} \rangle \langle \text{digit} \rangle$
 $\langle \text{digit} \rangle$

.....
 $\langle \text{extendedSubmissionCode} \rangle ::= \text{'ABS-'} \langle \text{submissionCode} \rangle \mid$
 $\langle \text{submissionCode} \rangle$

5. The following statements pertain to a university's course enrollment system, described using a predicate logic based programming language where facts and rules state what to do but not how to do it.

```

01 completed(zak, math101, a) .
02 completed(mike, math101, c) .
03 completed(sara, math101, b) .
04 completed(alex, math101, d) .
05 hasLicence(zak) .
06 hasLicence(sara) .
07 passedTheoryTest(zak, compsci201) .
08 passedTheoryTest(sara, compsci201) .
09 interestedInCourse(zak, compsci201) .
10 interestedInCourse(mike, compsci201) .
11 interestedInCourse(sara, compsci201) .
12 interestedInCourse(alex, compsci201) .
13 canEnroll(X, C)
    IF interestedInCourse(X, C) AND hasLicence(X) AND
    passedTheoryTest(X, C) .

```

Given the facts and rules in the code and references to clauses above:

- a. Give the name for type of programming language used [1]

Declarative programming language.

- b. Give one example of a fact. [1]

completed(zak, math101, a)

- c. Give one example of a rule. [1]

- d. Give one example of a goal to query the given knowledge base. [1]

canEnroll(X, compsci201).

- e. Give one example of resulting instantiation against the goal in d. above. [1]

X = zak , Sara

X = zak

X = Sara.

f. Create a rule that identifies students who are not eligible to enroll in a specific course because they:

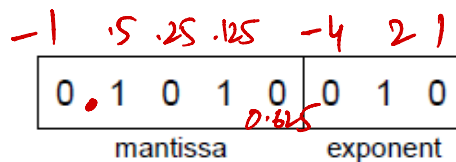
-  either have not passed the required theory test for the course,
-  or do not have a license.

ineligibleForEnrollment (X,C)

IF interestedInCourse (X,C) AND
NOT (HasLicense(X) AND passedTheoryTest (X,C)).

[3]

6. In any calculations, you must show how you obtain your answers.
 A binary number can also be written in a floating point representation as:



- (a) (i) Write down the denary value represented by the **exponent** in this example. [1]

+2

- (ii) Calculate the denary value of the binary number in this floating point representation. [2]

01010 010
 0.1010 $\times 2^2$
10.10
+2.5

- (b) Write, in binary, the largest number that can be represented in this floating point representation. [2]

01111 011 ✓
 0.1111 $\times 2^3$
111.1
+7.5

.25
 .03125
.28125

10.3
1010.01001

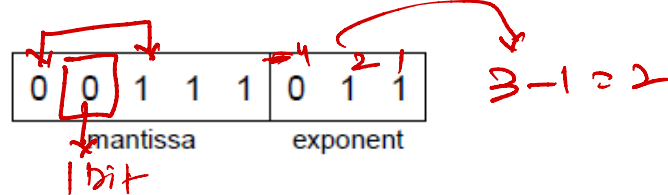
.25 .03125
 3 $\times 2 = 0.6$
 6 $\times 2 = 1.2$
 2 $\times 2 = 0.4$
 0.4 $\times 2 = 0.8$
 0.8 $\times 2 = 1.6$

3.8

11.11001
 .5
 .25
 .03125
0.78125

0.8 $\times 2 = 1.6$
 0.6 $\times 2 = 1.2$
 0.2 $\times 2 = 0.4$
 0.4 $\times 2 = 0.8$
 0.8 $\times 2 = 1.6$

(c) Another binary number is written in the same format as:



Write this FP number in the normalised form.

[2]

0.1110 010
 0.1110×2^2
 $+3.5$

(d) State why binary numbers are normalised.

[1]

Binary numbers should be normalised to ensure a consistent and efficient representation, avoid multiple representations of the same number, and minimise loss of precision during calculations.

7.

a. Explain why the number of machine cycles used by RISC and CISC to complete a single instruction is different.

[2]

RISC architecture uses simple, fixed length instructions that can be executed in a single machine cycle, making operations faster and more predictable.

CISC architecture use complex, variable length instructions that may perform several operations at once, often requiring multiple machines cycles to complete a single instruction.

b. An array processor is used in SIMD systems.

(i) Explain the term **SIMD** processor.

[2]

Single instruction multiple data or SIMD processor execute a single instruction simultaneously across multiple data points.

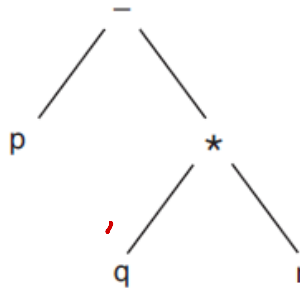
It processes large data sets faster by applying the same operation to several pieces of data at once which is highly useful for task like image processing, scientific calculations and multimedia applications.

(ii) Give one example of the type of computer used, for which a **SIMD** processor is most suitable.

[1]

Graphics Processing Unit (GPUs).

- c. Following binary tree describes operands p, q and r with operators – and *.



- (i) Write the reverse polish notation based on the binary tree above. [2]

..... p q r * -

- (ii) State the type of traversal that should be used to obtain the reverse polish notation from a binary tree. [1]

..... post order traversal (Left, Right, Post)

- (iii) An expression in reverse polish notation is stated as: t u * v + [1]
Showing how you obtained your answer, evaluate this expression, where:

t = 2, u = 3 and v = 10 [2]

..... 2 * 3 = 6 + 10

.....

3
2

10
6

 =

16

 ✓

- (iv) State data structure, other than binary tree, you used to evaluate RPN in c(iii). [1]

..... STACK

- (v) State a symbol, often used for infix mathematical notations, that is not required when working with RPN. [1]

..... (Brackets) parenthesis

8. Complete truth table below for AND gate and draw the symbol for the same. [2]

(a)

Input A	Input B	Output
0	0	0
0	1	0
1	0	0
1	1	1

AND gate symbol



(b) Table below list six Boolean laws. **Three** of them are correct. Shade the circles next to the three equations that are correct. [3]

Equation	Correct? (Shade three)
$A \cdot \bar{A} = 1$	☐
$A + B = \bar{\bar{A}} \cdot \bar{\bar{B}}$	☒
$A + 1 = 1$	☒
$A \cdot (A + B) = A$	☒
$A + (A \cdot B) = B$	☐
$A \cdot 1 = 1$	☐

$\bar{A}B = \bar{A} + \bar{B}$
De Morgan's

$A \cdot (A + B) = A$
 $A + (A \cdot B) = A$
↓
A

(c) Using the laws describing Boolean algebra, Simplify the following Boolean expression. [3]

$$(X + Y) \cdot (X + \bar{Y})$$

$$\begin{aligned}
 & (X + Y) \cdot (X + \bar{Y}) \\
 \Rightarrow & X \cdot (X + \bar{Y}) + Y \cdot (X + \bar{Y}) \\
 \Rightarrow & X + Y(X + \bar{Y}) \\
 \Rightarrow & X + Y \cdot \bar{Y} \\
 \Rightarrow & X + 0 = X
 \end{aligned}
 \left\{
 \begin{aligned}
 & (X + Y) \cdot (X + \bar{Y}) \\
 \Rightarrow & (X \cdot X) + (X \cdot \bar{Y}) + (Y \cdot X) + (Y \cdot \bar{Y}) \\
 \Rightarrow & X + (X \cdot \bar{Y}) + (X \cdot Y) \\
 \Rightarrow & X + X \cdot Y + \bar{Y} \\
 \Rightarrow & X + X \cdot 1 \\
 \Rightarrow & X
 \end{aligned}
 \right.$$

9. The TCP/IP protocol suite can be viewed as a stack of layers.

a. (i) Complete the stack by inserting the names of the missing layers below.

Application
Transport
Internet
Link

[3]

(ii) State how each layer of the stack is implemented.

[1]

Each layer of the stack is implemented in software with the exception of the link layer which is implemented in both software and hardware form. Each layer communicates with other layers above and below it.

b. A computer is currently running two processes:

 Process 1 is downloading and opening a web page in browser software.

 Process 2 is downloading an email using email client software.

Describe two tasks that the Transport layer performs to ensure that the incoming data is downloaded correctly.

[4]

1. segmentation and reassembly: the transport layer divides large data streams into smaller segments for easier and reliable transmission. At the receiving end, it reassembles the segments back into the original message in correct sequence.

2. error detection and correction: the transport layer checks incoming data segments for any transmission errors. If an error is detected, it requests transmission of the corrupted segment to ensure that complete and accurate data is delivered. -

c. Name a possible protocol that will be used by the Process 1.

[1]

http / https

d. Name a protocol that will be used by Process 2.

[1]

POP3

IMAP

